SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Rekha Mondal *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rekha Mondal (192521292)

Question 1: AI Drones for Emergency Medicine Delivery

Problem setup. The drone navigates a graph where nodes are locations or waypoints and edges are travel segments. Each edge has a variable cost g (time or energy) that depends on terrain and weather. The heuristic h(n) is the straight-line distance to the goal. Straight-line distance is admissible (it never overestimates, since no route can be shorter than a straight line), but it ignores floods, wind and blocked paths.

(i) Greedy Best-First Search (GBFS). GBFS expands the node with the smallest h(n) and ignores the cost already spent. The drone would head straight toward the hospital or relief camp, always choosing the waypoint that looks closest to the goal.

Strengths: it is very fast and needs little computation, which suits an urgent mission. It produces a first plan almost instantly and reacts quickly when new information arrives.

Weaknesses under uncertainty: it is neither optimal nor complete in graphs with loops. It may fly into a dead end, such as a flooded valley, because that waypoint is geometrically close to the goal. It ignores weather cost entirely, so it may choose a path through a storm cell simply because it is straight. It can also oscillate when blocked paths appear, and it carries no guarantee on delay or risk.

(ii) A*. A* expands the node with the smallest f(n) = g(n) + h(n). Here g(n) is the actual accumulated cost (including weather and terrain penalties) and h(n) is the estimated remaining distance. The g term keeps the search honest about the cost already paid, and the h term pulls the search toward the goal. This balance means A* avoids expensive detours that GBFS would take, and it will abandon a path whose real cost grows too high.

With an admissible and consistent heuristic, A* is complete and optimal. Straight-line distance is consistent when edge costs are at least the geometric distance, which holds if weather only adds cost. The price is higher memory and time, because A* keeps a larger frontier.

(iii) Impact of heuristic accuracy.

- *GBFS* depends entirely on the heuristic. If h is poor (for example, a river lies between the drone and the goal), GBFS is led astray with no cost term to correct it.
- *A** is more robust because g corrects heuristic errors. A more accurate h (say, one that includes known wind or terrain factors) shrinks the number of nodes expanded. If h is a gross underestimate (h = 0), A* degenerates into uniform-cost search: still optimal, but slow. If h overestimates, A* loses its optimality guarantee.

(iv) Dynamic changes. When a road is blocked, edges disappear or their costs jump. For GBFS, a blocked path means the greedy choice is invalidated and the drone must restart or backtrack, with no guarantee it won't hit another dead end. For A*, the previous g-values become stale, so the plan must be recomputed. The practical answer is replanning, using D* Lite or Lifelong Planning A* (LPA*), which reuse earlier search effort and repair only the affected part of the plan. Partial real-time updates also mean the drone's map is incomplete, so it should replan periodically or whenever new information arrives.

(v) Recommendation. I recommend **A* with a weather-aware heuristic and incremental replanning (D* Lite)**. The reasons:

1. **Optimality**: it minimizes combined delay and cost, not just distance.
2. **Safety**: g can include a risk penalty for storm zones, so dangerous routes are avoided even when short.
3. **Real-time**: the search can be bounded, for example with Weighted A* ($f = g + w \cdot h$ with w slightly above 1), trading a little optimality for speed.
4. **Dynamism**: incremental replanning handles blocked roads without a full restart.

GBFS is useful only as a fallback when computation time is almost zero, such as an emergency detour decision. For a life-critical delivery, A*'s guarantees outweigh its modest extra computation.

Question 2: Smart City Traffic Signal Optimization

Formulation as an optimization problem.

- **State:** a complete assignment of signal timings (cycle length, green split and offset) for every intersection.
- **Decision variables:** the timing parameters for each of the hundreds of intersections.
- **Objective function:** minimize $f(x) = w_1 \cdot$(total waiting time) + $w_2 \cdot$(fuel consumption) + $w_3 \cdot$(congestion index), with weights set by city policy.
- **Constraints:** minimum and maximum green times, pedestrian crossing times, and safety clearances.
- **Neighbors:** a new state obtained by changing one or a few timing values.

Why traditional search is inefficient. If each intersection has even 20 possible timing settings, 300 intersections give 20^{300} states. BFS, DFS and A* try to find a path from a start state to a goal state, but here there is no single goal state. We only want the best-scoring configuration. The state space is astronomically large, the evaluation is costly (it requires a traffic simulation), and traffic patterns change, so any precomputed optimum soon becomes stale.

(i) Hill Climbing. Start with the current timing plan. Generate neighbors (for example, extend one intersection's green phase by 5 seconds). Move to the neighbor with the best objective value, and repeat until no neighbor improves the score. It uses very little memory (only the current state) and is fast and simple, so it can run continuously as a live tuner.

Limitations*:* it is greedy and has no memory. It cannot escape poor regions and its result depends heavily on the starting point.

(ii) Local maxima, plateaus and ridges.

- **Local maximum (here, a local minimum of cost):** a timing plan where every small change makes things worse, but a much better plan exists elsewhere. Example: synchronizing signals along one avenue reduces delay there, but any single change breaks the "green wave." A globally better plan would restructure the whole district.
- **Plateau:** a region where small changes leave the objective unchanged. Example: when traffic is light, changing a green split by a few seconds does not change total waiting time, so the algorithm has no gradient to follow and wanders.
- **Ridge:** a narrow improving path that single-variable moves cannot follow. Example: improving flow requires changing two adjacent intersections' offsets together. Changing either one alone worsens things, so hill climbing sees no improvement.

(iii) Simulated Annealing and Genetic Algorithms.

Simulated Annealing (SA) accepts worse moves with probability e^(−ΔE/T). At high temperature T it explores widely, and as T cools it behaves more like hill climbing. This lets it escape local optima and cross plateaus. With a slow enough cooling schedule it converges toward the global optimum, and it is easy to implement on top of hill climbing.

Genetic Algorithms (GA) maintain a population of timing plans. Fitness is the negative of the cost, selection favors good plans, crossover combines good sub-plans (for example, one district's timing from parent A and another's from parent B), and mutation introduces random changes. GA explores many regions at once, handles ridges through crossover, and parallelizes well, since the population can be evaluated on many machines.

(iv) Recommendation. I recommend a **hybrid approach: a Genetic Algorithm for periodic global optimization, with Simulated Annealing or hill-climbing for fast local adjustment**.

- **Scalability**: GA works on a population and is highly parallelizable, and the city can be decomposed into districts optimized semi-independently.
- **Adaptability:** when traffic patterns shift, the previous best plans seed the next population (warm start), so the system adapts quickly instead of starting over. SA's local fine-tuning then handles minute-by-minute changes.
- **Robustness:** both methods avoid the local-optimum trap that pure hill climbing falls into.

Pure SA is a good lightweight alternative for smaller deployments. For hundreds of intersections with changing demand, the hybrid gives the best balance of solution quality and real-time responsiveness.

Question 3: Autonomous Mars Rover

(i) Why online search is required. Offline search computes a complete plan before acting, which assumes the whole map and all action outcomes are known. The rover has neither. The terrain is unknown, sensors see only a limited radius, and communication with Earth has a delay of several minutes, so remote control is impractical. An *online search agent* interleaves planning and acting: it takes a step, observes, updates its knowledge and then plans again. This suits situations where exploration itself is part of the task. Offline planning would be both impossible (no map) and dangerous (a plan built on assumptions may drive the rover into a crater).

(ii) Challenges of partial observability and dynamic environments.

- **Partial observability:** the rover sees only nearby terrain, so it cannot tell whether a path ahead leads to the goal or a dead end. Sensor noise makes a shadow look like a crater, or a rock look like flat ground. The rover must therefore estimate its state, using a belief state rather than a certain one.
- **Dynamic environments:** dust storms alter visibility, sand shifts, and terrain that was safe may become unstable. Previous observations can go stale.
- **Irreversible actions:** falling into a crater or getting wheels stuck may be unrecoverable, so the agent must treat safety as a hard priority rather than just a cost.
- **Resource limits:** limited power, computation and sensing mean exploration must be efficient.

(iii) Building and updating the internal model. The rover maintains a map, typically an occupancy grid or a graph. Each cell stores a status (free, obstacle, hazard, unknown) and often a probability. At every step it:

1. Senses its surroundings (cameras, LIDAR, wheel odometry).
2. Localizes itself (for example, with SLAM, or a Kalman or particle filter) to correct for wheel slip.
3. Updates the map with Bayesian reasoning: new observations raise or lower the probability that a cell is blocked.
4. Records visited states and the results of actions (for instance, a cost-to-goal table as in LRTA*, Learning Real-Time A*).
5. Re-plans a path over the updated model.

Unknown cells are treated cautiously, either as obstacles or with an added risk cost, until observed.

(iv) Comparison of strategies.

- **Uninformed search (BFS/DFS/UCS):** these need a known state space. In an unknown environment they explore blindly, waste energy, and DFS-style physical backtracking is costly because the rover must actually drive back.
- **Informed offline search (A*, Greedy):** efficient with a good heuristic, but they assume a known map and deterministic outcomes. A* plans a perfect route over a map that may be wrong.

- **Online search (LRTA*, online DFS, D* Lite):** these act with only local knowledge, update their heuristic or map as they go, and can be informed by a heuristic such as distance to the sample site. They are not guaranteed optimal, but they are complete in safely explorable spaces and are designed for exactly this situation. Their cost is measured as the actual distance traveled (competitive ratio), not the planning time.

(v) Justified approach. I recommend an **online, informed, incremental replanning approach: D* Lite (or LRTA*) running on a continuously updated probabilistic map, with a safety layer.**

- **Uncertainty:** the probabilistic map and belief updating handle noisy, partial data.
- **Adaptability:** D* Lite efficiently repairs the plan when new obstacles are discovered, reusing earlier computation.
- **Safety:** hazard cells get very high costs, and a conservative rule (never enter an unverified cell if a safe alternative exists) is enforced, with the ability to stop and wait for human input.
- **Efficiency:** a goal-directed heuristic reduces wandering compared with blind exploration.

This combination gives safe, goal-directed progress without needing a complete map.

Question 4: University Exam Timetabling as a CSP

Formulation. A CSP is defined by variables, domains and constraints.

(i) Variables, domains and constraints.

Variables: each exam E_i has a time slot variable T_i and a room variable R_i (and optionally an invigilator variable F_i). Example: T(Math101), R(Math101).

Domains: $T_i \in$ {all available slots, such as Day1-Morning, Day1-Afternoon, …}. $R_i \in$ {exam halls that can hold the number of enrolled students}. Special accommodations can shrink domains. For instance, a student who needs a ground-floor accessible room limits the domain of that exam's room.

Constraints:

1. **No student clash:** for any two exams E_i and E_j with a common student, $T_i \neq T_j$. (Binary constraint.)
2. **Hall capacity and exclusivity:** if $T_i = T_j$ then $R_i \neq R_j$, and students enrolled in $E_i \leq$ capacity(R_i). Since the number of halls is limited, the number of simultaneous exams cannot exceed the number of halls.
3. **Precedence:** if subject A must precede B, then $T_a <$ T_b.
4. **Faculty availability:** T_i must be in the available set of the invigilator or examiner, and one faculty member cannot invigilate two exams at once.
5. **Special accommodations:** extra time requires longer slots or following-slot buffers, and accessibility needs restrict rooms.

Hard constraints must hold, while soft preferences (for example, avoiding three exams in a day for one student) can be optimized afterward.

(ii) Backtracking search. Backtracking is depth-first search that assigns one variable at a time and checks consistency after each assignment. Example: assign T(Math101) = Day1-Morning. Then assign T(Physics101). If it conflicts with Math101 through shared students, try the next value. If a variable has no consistent value left, the algorithm backtracks to the previous variable and tries another value. It prunes whole branches early, unlike brute-force enumeration. But plain backtracking can thrash, repeatedly failing for the same underlying reason deep in the tree.

(iii) Constraint propagation and forward checking. After each assignment, forward checking removes inconsistent values from the domains of unassigned neighboring variables. If Math101 takes Day1-Morning, that slot is deleted from the domain of every exam sharing students with it. If any domain becomes empty, the algorithm backtracks immediately, detecting failure far earlier. Stronger methods include arc consistency (AC-3), which propagates reductions further. Heuristics complement this: *MRV* (choose the variable with the fewest remaining values, so failures surface early), the *degree heuristic* (choose the exam involved in the most constraints), and *LCV* (try the value that rules out the fewest options for neighbors).

(iv) Real-world challenges and best strategy.

Challenges: thousands of courses and students make the constraint graph huge and dense. Constraints change late (a hall becomes unavailable), and perfect satisfaction may be impossible (over-constrained), requiring relaxation. There are also competing soft goals such as fairness and student spacing.

Best strategy for scalability: use **backtracking with MRV + degree + LCV, forward checking or AC-3, and conflict-directed backjumping, combined with local search (min-conflicts) for repair**. Complex instances can be decomposed using graph colouring, in which exams are nodes and edges connect exams sharing students, so a colouring gives conflict-free slots. For very large instances, generate a good schedule with heuristics, then improve it with min-conflicts or simulated annealing, which repairs a nearly valid timetable quickly when constraints change. This hybrid keeps the guarantees of systematic search where feasible and gains the speed and flexibility of local search where it is not.

Question 5: Real-Time Strategy Game AI (Minimax and Alpha-Beta)

(i) How Minimax models the problem. Minimax treats the game as a two-player, zero-sum adversarial search. States are game positions. MAX is the AI and MIN is the human player. Players alternate turns, and the tree grows as each player's legal moves are expanded. Leaf (terminal or depth-limited) states receive a utility value. Minimax values propagate upward: MAX nodes take the maximum of their children, and MIN nodes take the minimum, because the algorithm assumes the opponent plays optimally against the AI. The AI then picks the move at the root with the highest minimax value. This gives the best guaranteed outcome against a perfect opponent.

(ii) Computational challenges. The time complexity is $O(b^m)$, where b is the branching factor and m is the depth. In a real-time strategy game, b is huge (many units, each with many actions, so possibly hundreds or thousands of joint moves), and the game runs for a long time. Even b = 30 and depth 10 gives about 6×10^{14} nodes, far beyond a time limit of milliseconds. Additional problems are that real-time play makes states change continuously (not turn-based), partial observability ("fog of war") hides enemy positions, evaluating each state is expensive, and memory use grows with depth. A full search to terminal states is infeasible, so depth must be limited.

(iii) How Alpha-Beta pruning improves efficiency. Alpha-Beta maintains two bounds: α is the best value MAX can already guarantee along the current path, and β is the best value MIN can guarantee. If at any node $\alpha \geq \beta$, the remaining children cannot influence the final decision, because the opponent would never allow this line, so they are pruned.

Reasoning example: suppose MAX has already found a move worth 5 ($\alpha = 5$). While exploring another move, MIN finds a reply worth 3. MIN can hold MAX to 3 or less in that branch, which is already worse than 5, so MAX will never choose it. The other replies in that branch need not be examined.

Alpha-Beta returns exactly the same result as Minimax, so there is no loss in quality. Its efficiency depends on move ordering. In the worst case, it examines all nodes, $O(b^m)$. With perfect ordering, it examines about O(b^(m/2)) nodes, which effectively doubles the searchable depth. Good ordering uses captures or high-value moves first, killer moves, and iterative deepening, where the best move from the previous iteration is tried first.

(iv) Evaluation function design. Since the AI cannot search to the end of the game, it uses a heuristic evaluation function Eval(s), typically a weighted linear sum: $Eval(s) = w_1f_1 + w_2f_2 + \ldots + w_nf_n$. Suggested features:

- **Material/army strength:** total unit value, health and upgrades (mine minus the opponent's). It reflects combat capability.
- **Economy:** resources gathered, income rate and worker count, because economy converts into future strength.
- **Map and territory control:** number of bases, strategic points and vision held, since they give positional advantage.
- **Tech and production capacity:** unlocked technologies and buildings, indicating long-term potential.
- **Threat and safety:** proximity of enemy forces to my base, vulnerability of key units, and whether my commander is exposed.
- **Mobility/tempo:** the number of useful moves available, which rewards flexibility.

Weights can be tuned through self-play or machine learning, and they may change by game phase (economy matters early, army matters late). The function must be fast to compute and correlate well with winning chances.

(v) Balancing optimality and real-time performance. I recommend:

1. **Iterative deepening Alpha-Beta with a time budget:** search depth 1, 2, 3… until time runs out and return the best move from the last completed depth, so an answer is always ready.
2. **Move ordering and a transposition table** to cache repeated states and prune more.
3. **Abstraction and move reduction:** group units into squads and consider high-level actions (attack, defend, expand) to cut the branching factor.
4. **Quiescence search** to avoid evaluating unstable positions mid-battle.
5. **Selective deepening or forward pruning** (for example, beam search) to focus on promising lines, accepting a small loss of optimality.
6. **Alternatives such as Monte Carlo Tree Search (MCTS)** for extremely large branching factors, since it samples rather than enumerates and can stop at any time.

This approach gives near-optimal decisions within strict time limits, adaptive depth based on available computation, and strong play without exhaustive search.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understandin g of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned , logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand